Lab 2 Evidence Report: Secure Student Records Portal

ICS570 Cybersecurity Essentials, Module 3: Identity and Access Control. Group 3: Henry Nii-Amarh Olletey, Kobina Adzeyena Opei Junior, Eric Sunu and Clifford Yeboah. 17 October 2026.

1. Setup

The portal signs users in with passkeys and checks every protected request twice: first that a verified passkey session exists, then that the policy allows this user this action on this record. All 16 tests passed.

- **Stack:** Node.js 22, Express 5, express-session, `@simplewebauthn/server` and `/browser` v14, SQLite (`node:sqlite`). Source, lock file, seed data and evidence: https://github.com/ASU-MICS-2028/cs-group-3-assignment-1
- **Passkeys:** RP ID `localhost` and origin `http://localhost:3000` are checked on every registration and sign-in, with user verification required. Challenges are single use and expire after two minutes.
- **Enrollment:** a passkey can be registered only after entering a one-time code (SHA-256 hash stored, single use, 24-hour expiry).
- **Test run:** `npm run evidence` on macOS 26.6 with Google Chrome 155 and Node 22.22, against a freshly seeded database. Chrome's built-in virtual authenticator creates the passkeys and performs the same WebAuthn ceremony as Touch ID; the server verifies it the same way. T1 and T2 were also done by hand in Microsoft Edge with Touch ID; the server log shows both verified (200) and the record loaded (Figure 3).

2. Results

Every permitted action succeeded and every prohibited one was refused by the server (full output in `evidence/results.md`).

Test	Identity	Request or action	Expected	Observed
T1	S001	Enter one-time code, register a passkey	Credential stored for S001	200; `passkeys` row for S001 with its public key
T2	S001	Sign in with the passkey	Session created	200; `GET /me` returns S001, student
T3a	S001	Sign out, get a fresh challenge, resend the T2 response	Rejected, no session	400 challenge mismatch; `/me` 401
T3b	S001	Alter one character of the signature in a live sign-in	Rejected, no session	400 signature not verified; `/me` 401
T4	none	`GET /students/S001` with no session	Denied	401 Not signed in
T5	S001	`GET /students/S001`	Allowed	200; own profile and grade
T6	S001	`GET /students/S002`, by URL and by fetch	Denied	403 Forbidden, both
T7	S001	`POST /courses/C101/grades/S002`, body claims role lecturer	Denied, grade unchanged	403; grade A before and after
T8	L001	Change S001's C101 grade in the UI	Allowed	200; B+ to A, audit line written
X1	L001	`POST /courses/C102/grades/S002` (not L001's course)	Denied	403; grade still B
X2	L001	`GET /students/S002` (takes C101 and C102)	C101 part only	200; only C101 returned
X3	any	Reuse S001's enrollment code	Rejected	400
X4	none	`GET /webauthn/register/options` without a code	Denied	401
X5	S001	Sign out, then `GET /students/S001`	Denied	401
X6	L001	Registrar removes L001 from C101; L001, still signed in, retries T8	Denied on next request	403; record read also 403
X7	S001	Send one valid sign-in response twice in parallel	Only one succeeds	200 and 400

3. Selected evidence

Signed in as S001, we sent two requests the UI never offers. The server refused both, and S002's grade did not change.



Figure 1. T6 and T7: requests sent from S001's signed-in page, the same as typing them into the DevTools console. The body's claim to be a lecturer is ignored.

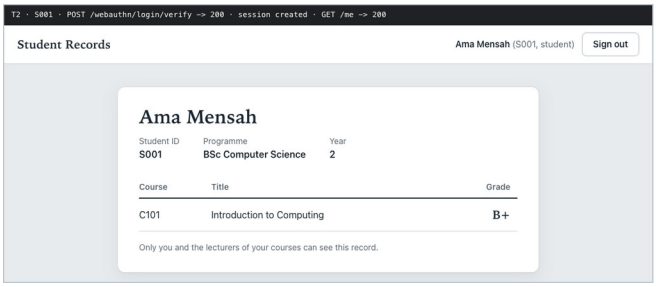
The checks that denied them. `requireAuth` takes the user and role from the server-side session and the database, never from the request. The route then asks the policy, which denies unless a rule allows:

```
// server.js
app.post('/courses/:courseId/grades/:studentId', requireAuth, (req, res) => {
  if (!policy.canEditGrade(req.user, courseId, studentId)) return deny(req, res, 'grade.update', ...);

// policy.js
function canReadStudent(user, studentId) {
  if (user.role === 'student') return user.id === studentId;            // own record only
  if (user.role === 'lecturer') return sharesCourse(user.id, studentId); // their courses only
  return false;
}
function canEditGrade(user, courseId, studentId) {
  return user.role === 'lecturer' && teaches(user.id, courseId) && isEnrolled(studentId, courseId);
}
```

Accountability. Each decision is recorded in `audit_log` (from `evidence/audit-log.txt`, timestamps omitted):

```
S001               student.read     S002       denied   role=student
S001               grade.update     C101/S002  denied   role=student
L001               grade.update     C101/S001  allowed  B+ -> A
registrar-console  course.unassign  L001/C101  allowed  1 row removed
L001               grade.update     C101/S001  denied   role=lecturer
```



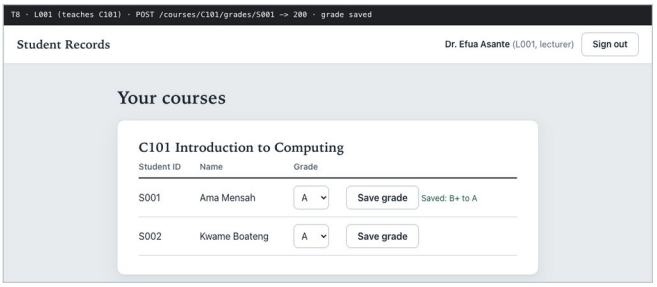


Figure 2. Left: T2 and T5, S001's own record after a verified passkey sign-in. Right: T8, L001 saves S001's C101 grade and the server returns 200.

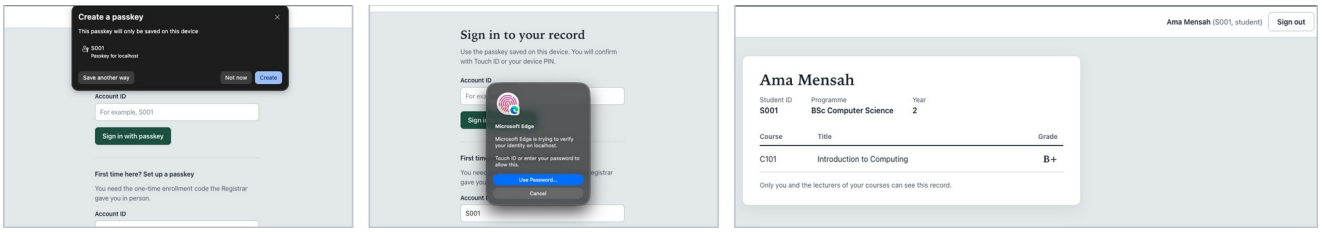


Figure 3. T1 and T2 by hand with Touch ID in Microsoft Edge: the browser's "Create a passkey" dialog for S001 on localhost, the Touch ID prompt, and S001's record after signing in with that passkey.

4. Access matrix

The `lecturer_courses` and `enrollments` tables form the authorization table (L07 slides 12–14). Roles give the coarse rule (RBAC); ownership and course assignment narrow it, as attribute checks.

Subject	S001 record	S002 record	C101 grades	C102 grades
S001 (student)	Read	None (T6)	None (T7)	None
S002 (student)	None	Read	None	None
L001 (lecturer, C101)	Read C101 part	Read C101 part (X2)	Read, write (T8)	None (X1)
Visitor, no session	None (T4)	None	None	None

No user can administer accounts through the portal. Course assignment, enrollment codes and passkey revocation happen only in a local Registrar console (`npm run admin`), which keeps those privileged functions off the web.

5. Security interpretation

A. Authentication vs authorization. T2 proved only that the person signing in holds S001's private key: the server checked a fresh challenge, origin, RP ID and signature before creating a session. That answers "who are you?". T6 and T7 ask a different question, "what may S001 do?", and the server answers it again on every request. `requireAuth` in server.js admits only a session created by a verified passkey (T4 and X5 returned 401). policy.js then decides each action and denies by default: `canReadStudent` refused S002's record (T6, 403), and `canEditGrade` refused the grade change because S001 is not the lecturer of C101 (T7, 403, grade unchanged), even though the request body claimed the role "lecturer".

B. Residual weakness. The weakest point is recovery. If a student loses their laptop, the only way back is a new enrollment code, and whoever holds that code can register their own passkey on the account. X3 shows a used code is refused, but a freshly issued one works, so an attacker can bypass phishing-resistant sign-in by targeting the Registrar. A proportionate fix is to issue replacement codes only after an in-person ID check, notify the user whenever a passkey is added, and let users register a backup passkey at enrollment.

C. Transfer to SLS ProSit 2. X6 shows why ProSit 2's Finance-to-Procurement mover still had payment rights after six weeks. When the Registrar removed L001 from C101, L001's very next grade change returned 403, although L001 was still signed in. That works because our policy reads the `lecturer_courses` table on every request instead of trusting what was granted at login. At SLS, nothing linked the employee's access to their current role, so the old Finance right stayed. Authentication was never the problem there: the employee is who they claim to be. The lesson is that a role change must update the authorization data, triggered by an HR event rather than someone remembering, and periodic access reviews should catch anything missed. Stronger login alone, as management proposed, would not have removed that access.

6. AI and tools declaration

Claude (Anthropic) was used to generate the code for the portal, including its web interface, server-side checks and automated test runner, and to help aggregate our ideas and thoughts and word this report. Group 3 reviewed the material and takes responsibility for the work presented.